

A Portable ML-KEM Implementation in ISO C11 with Reusable Decapsulation Pools

Kostiantyn Zaverailo

Abstract

This paper presents a portable implementation of the ML-KEM post-quantum key encapsulation mechanism standardized in FIPS 203. The implementation is written in ISO C11 with an emphasis on portability, explicit memory management, and architectural clarity across userspace and Linux kernel-oriented environments.

A pool-based decapsulation architecture is introduced to reduce repeated allocation and reinitialization overhead during decapsulation operations. The design employs fixed-size preallocated memory slots for sensitive decapsulation data that allow repeated reuse of allocated memory regions between operations after secure clearing of previous slot contents. Constant-time ciphertext validation logic is also employed to support efficient and predictable decapsulation behavior.

The implementation was evaluated using multiple testing methodologies, including NIST Known Answer Tests (KATs), stress testing, malformed ciphertext validation, AddressSanitizer (ASAN), ThreadSanitizer (TSAN), Valgrind-based memory analysis, and dupect-based timing leakage analysis under both GCC and Clang toolchains on x86-64 systems.

The project is intended as a portable engineering-oriented ML-KEM implementation and experimental platform for evaluating reusable decapsulation architectures, portability-oriented ISO C11 design strategies, and practical constant-time testing workflows.

Keywords: ML-KEM, Post-Quantum Cryptography, ISO C11, Decapsulation, Constant-time, Reusable decapsulation pools, Explicit memory management

1. Introduction

Post-quantum cryptography has become an increasingly important area of research and engineering due to the long-term security risks posed by large-scale quantum computers to classical public-key cryptographic systems. Therefore, the National Institute of Standards and Technology (NIST) standardized the Kyber algorithm under the new name ML-KEM in FIPS 203 as a post-quantum key encapsulation mechanism intended for general cryptographic use[1].

Practical deployments of ML-KEM require implementations that are not only functionally correct, but also portable, maintainable, and suitable for integration into different execution environments, including userspace applications and operating system kernels. At the same time, implementations that process sensitive information must carefully consider memory management behavior, concurrency, and constant-time execution characteristics to reduce the risks of unintentional information leakage and minimize opportunities for attackers to exploit implementation-level weaknesses.

This work presents a portable ISO C11 implementation of ML-KEM designed with an emphasis on architectural clarity, explicit memory management, and portability-oriented design[4]. The implementation supports both userspace-oriented and Linux kernel-oriented integration strategies while attempting to minimize dependencies on platform-specific compiler extensions or architecture-specific optimizations. The use of standardized ISO C11 constructs is also intended to simplify adaptation of the implementation to additional environments with minimal implementation changes.

In addition to the core implementation, this work introduces a pool-based decapsulation architecture intended to reduce repeated allocation and reinitialization overhead during decapsulation operations. The design uses fixed-size preallocated decapsulation slots that enable repeated reuse of allocated memory regions while securely clearing slot contents between decapsulation operations.

The implementation was evaluated using multiple testing methodologies, including NIST Known Answer Tests (KATs), stress testing, malformed ciphertext validation, AddressSanitizer (ASAN), ThreadSanitizer (TSAN), Valgrind-based memory analysis, and duddect-based timing leakage analysis under both GCC and Clang toolchains on x86-64 systems.

The primary contributions of this work are:

- a portable ISO C11-oriented ML-KEM implementation;
- a pool-based memory reuse approach for ML-KEM decapsulation;
- a portability-oriented design approach supporting both userspace and Linux kernel integration;
- extensive correctness, robustness, and timing-oriented testing workflows.

2. General Architecture and Testing Methodology

2.1 Overview of ML-KEM

ML-KEM is a lattice-based post-quantum key encapsulation mechanism standardized in FIPS 203[1]. The algorithm provides three primary operations: key generation, encapsulation, and decapsulation. Internally, ML-KEM relies on structured polynomial arithmetic, Number Theoretic Transform (NTT)-based computations, and SHA3/SHAKE cryptographic primitives to derive shared secrets and process ciphertexts efficiently. The standard defines multiple parameter sets intended to provide different security levels while maintaining practical performance characteristics for real-world deployment scenarios.

The overall implementation architecture is illustrated in Figure 1. The design separates persistent cryptographic objects, temporary working contexts, reusable decapsulation resources, and lower-level cryptographic processing layers in order to simplify portability and security-oriented memory management.

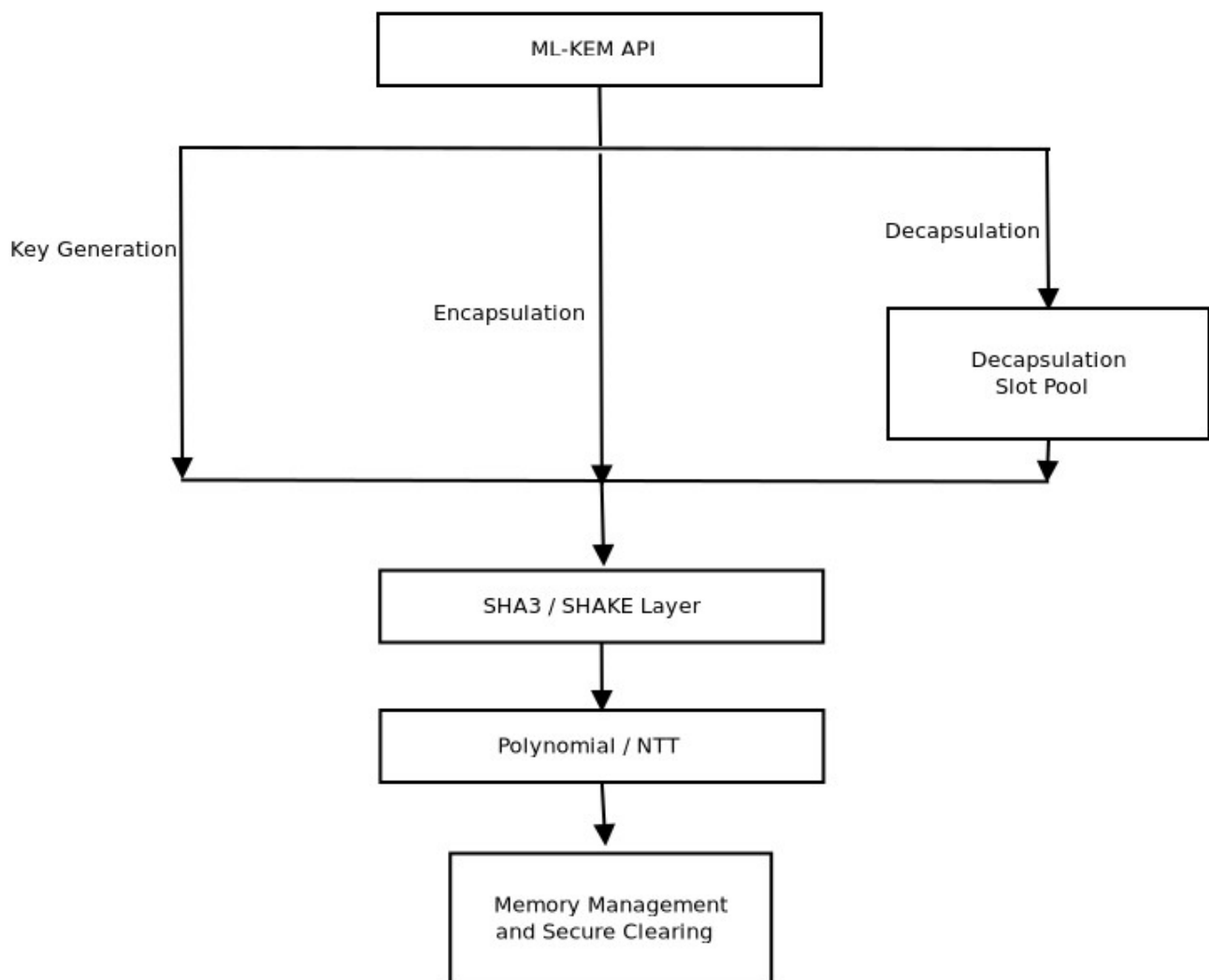


Figure 1. High-level architecture of the ML-KEM implementation

2.2 Implementation Architecture

The implementation is written in ISO C11 with an emphasis on portability, explicit memory management, and architectural clarity. The design attempts to minimize reliance on compiler-specific extensions and architecture-dependent optimizations in order to simplify adaptation to multiple execution environments.

The implementation architecture supports userspace-oriented execution, Linux kernel-oriented integration strategies, and portability-oriented adaptation to other execution environments compatible with ISO C11 semantics. Persistent cryptographic objects, temporary working contexts, and reusable decapsulation resources are separated into dedicated structures in order to simplify memory ownership, lifecycle management, and secure handling of sensitive intermediate data. Sensitive temporary buffers and reusable decapsulation slots are securely cleared after use before memory regions are reused or returned to the pool.

A reusable pool-based decapsulation design is employed to reduce repeated allocation and initialization overhead during decapsulation operations. The architecture uses preallocated fixed-size slots that may be reused between operations after secure clearing of sensitive intermediate data.

2.3 Security and Constant-Time Design Considerations

Because ML-KEM processes sensitive cryptographic material, the implementation attempts to reduce risks associated with timing-dependent behavior, cache-related information leakage, and improper handling of temporary sensitive data.

Constant-time oriented techniques are used in several security-critical execution paths, including ciphertext validation and shared-secret selection logic during decapsulation operations. The implementation attempts to avoid secret-dependent branching and secret-dependent memory access patterns in selected critical code paths in order to reduce opportunities for timing- and cache-based side-channel leakage.

The implementation also employs explicit clearing of temporary sensitive buffers and reusable decapsulation slots after use before memory regions are reused or returned to the decapsulation pool.

2.4 Testing Methodology

Multiple testing methodologies were used to evaluate implementation correctness, robustness, and timing-related behavior. Functional correctness was validated using NIST Known Answer Tests

(KATs) derived from the NIST ACVP test vector repository [2] and repeated end-to-end encapsulation and decapsulation testing.

Additional robustness testing included malformed ciphertext validation, multithreaded stress testing, AddressSanitizer (ASAN), ThreadSanitizer (TSAN), and Valgrind-based memory analysis [6], [8]. Timing-related behavior was additionally evaluated using duect-based statistical leakage testing under both GCC and Clang toolchains on x86-64 systems[5],[6],[7].

3. ISO CII-Oriented Portability Design

3.1 Portability-Oriented Design Goals

The implementation was designed with an emphasis on portability, architectural clarity, and explicit control over memory management behavior across multiple execution environments. ISO CII semantics were intentionally prioritized[3] in order to minimize reliance on compiler-specific extensions, platform-dependent behavior, and architecture-specific optimizations. The design also attempts to simplify adaptation of the implementation to userspace-oriented, Linux kernel-oriented, and other ISO CII-compatible environments with minimal structural modification. In addition, portability-oriented design choices were intended to support more predictable validation and testing workflows across different compiler toolchains and runtime environments.

3.2 Separation of Platform-Independent and Platform-Specific Logic

The implementation separates platform-independent cryptographic processing logic from platform-specific integration components in order to simplify portability across different execution environments. Core ML-KEM operations, including polynomial arithmetic, SHA3/SHAKE processing, and decapsulation logic, are reused across both userspace-oriented and Linux kernel-oriented configurations with minimal modification.

Platform-specific functionality such as memory allocation, randomness acquisition, and synchronization primitives is isolated into dedicated integration layers. This separation-oriented design approach reduces platform coupling and simplifies adaptation of the implementation to additional ISO CII-compatible environments.

3.3 Explicit Memory Management and Reusable Context Design

The implementation employs an explicit memory management approach intended to simplify ownership tracking, lifecycle management, and secure handling of sensitive cryptographic data. Persistent cryptographic objects, temporary working contexts, and reusable decapsulation resources are separated

into dedicated structures in order to improve architectural clarity and reduce unintended memory reuse behavior.

A reusable pool-based decapsulation design is used to reduce repeated allocation and reinitialization overhead during decapsulation operations through fixed-size preallocated slots. Sensitive temporary buffers and reusable slot contents are securely cleared before memory regions are reused or returned to the decapsulation pool.

3.4 Portability and Constant-Time Engineering Tradeoffs

The implementation prioritizes portability-oriented and predictable behavior over aggressive architecture-specific optimization strategies. Excessive platform-dependent tuning and compiler-specific optimizations were intentionally minimized in order to simplify portability validation and reduce implementation complexity across different environments. Because constant-time behavior may be affected by compiler transformations, the implementation was additionally evaluated under both GCC and Clang toolchains on x86-64 systems using dudefect-based timing leakage testing[5],[6],[7]. The use of standardized ISO C11 semantics was also intended to improve consistency of implementation behavior across multiple compiler and runtime configurations.

3.5 Modular Organization and Optional Low-Level Optimization Integration

The implementation employs a modular internal structure in which performance-critical operations, including NTT-related processing, are separated into dedicated implementation modules.

Although the current implementation is written entirely in ISO C11, the modular organization was intentionally designed to simplify potential integration of architecture-specific low-level optimizations when required. In particular, isolated computational modules may be selectively replaced or extended with assembly-oriented implementations without requiring major modification of higher-level ML-KEM logic.

This approach allows the implementation to preserve portability-oriented design principles while still supporting future optimization-oriented experimentation.

4. Pool-Based Decapsulation Architecture

4.1 Motivation for Reusable Decapsulation Pools

ML-KEM decapsulation repeatedly processes sensitive temporary data and may require frequent allocation, initialization, and clearing of intermediate working memory regions. Repeated dynamic memory management operations may introduce additional overhead, reduce predictability of memory behavior, and complicate concurrent execution scenarios under intensive decapsulation workloads. In order to reduce repeated allocation and reinitialization costs, the implementation employs a reusable pool-based decapsulation architecture using fixed-size preallocated slots[4].

The design is also intended to simplify memory lifecycle management and provide more predictable reuse behavior for sensitive temporary decapsulation resources. Such an approach may also be beneficial in resource-constrained or system-oriented environments where excessive dynamic allocation behavior is undesirable due to limited memory resources or allocation-related overhead.

4.2 Pool Organization and Slot Lifecycle

The decapsulation architecture employs a reusable pool structure consisting of multiple fixed-size decapsulation slots allocated during initialization[4]. Each slot stores temporary decapsulation-related data required during ciphertext processing and shared-secret derivation.

During decapsulation, an available slot is atomically acquired from the pool, used for temporary processing operations, securely cleared after completion, and then returned to the reusable slot pool for subsequent operations. The reusable slot lifecycle employed during decapsulation is illustrated in Figure 2. This reusable lifecycle-oriented design reduces repeated allocation behavior and provides more predictable management of sensitive temporary decapsulation resources.

The reusable slot approach is additionally simplified by the structure of the ML-KEM algorithm itself, where ciphertexts and several intermediate decapsulation-related data structures employ fixed and predictable sizes for a selected parameter set. As a result, securely cleared memory regions may be repeatedly reused for subsequent decapsulation operations without requiring variable-sized allocation behavior or complex memory layout management.

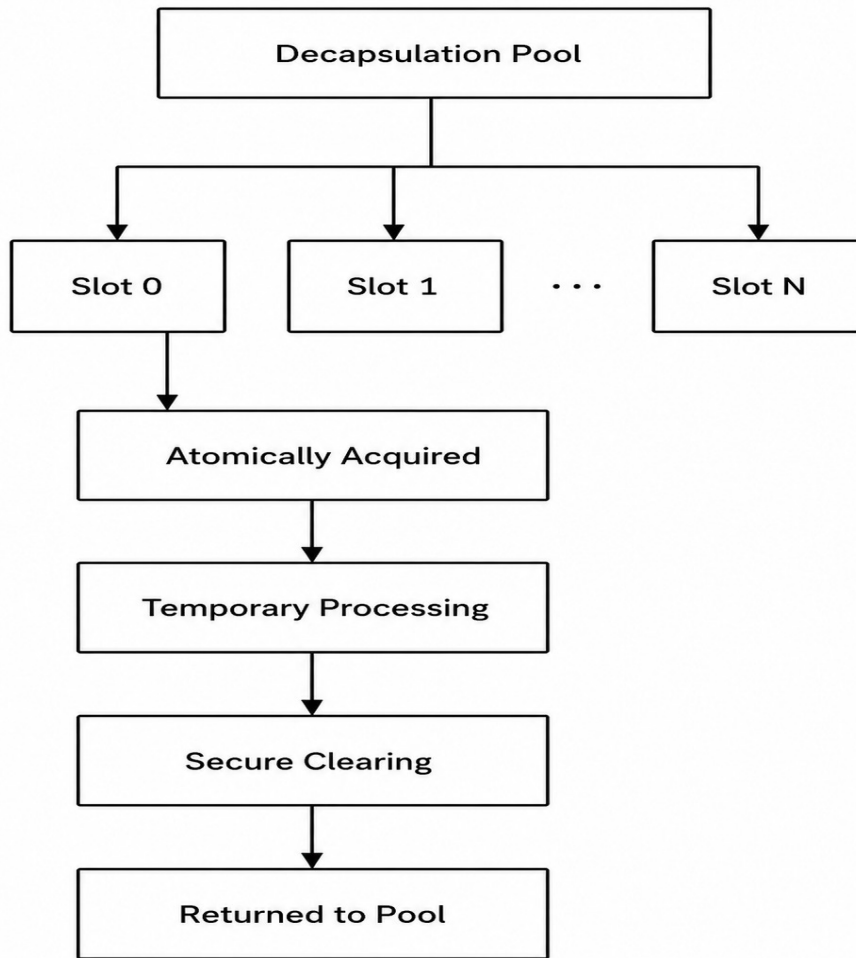


Figure 2. Decapsulation pool and reusable slot lifecycle

4.3 Decapsulation Workflow

During decapsulation, the implementation first validates input parameters and atomically acquires an available reusable slot from the decapsulation pool for temporary processing operations. The ciphertext is then processed in order to recover the intermediate message representation, after which re-encapsulation is performed to reconstruct the expected ciphertext and derive the candidate shared secret K_{bar} .

In parallel, an alternative fallback secret is derived from the stored secret value z and the received ciphertext in accordance with the ML-KEM decapsulation procedure[1]. Final shared-secret selection is performed using constant-time oriented comparison and selection logic intended to reduce risks associated with timing-dependent behavior during ciphertext validation and fallback handling.

4.4 Concurrency and Reuse Considerations

The reusable decapsulation architecture was designed with consideration for concurrent execution scenarios involving multiple simultaneous decapsulation operations. Available decapsulation slots are atomically acquired and returned to the reusable pool in order to reduce risks associated with concurrent access to temporary shared resources.

The use of fixed-size preallocated slots additionally reduces repeated interaction with dynamic memory allocators during intensive decapsulation workloads and simplifies predictable reuse behavior across repeated operations. The architecture also requires additional memory to be allocated during initialization in order to provision the reusable decapsulation pool with a sufficient number of preallocated slots for subsequent operations.

Multithreaded stress testing was also performed in order to evaluate correctness and stability of the reusable pool design under concurrent execution conditions.

4.5 Security Considerations of Pool Reuse

Because reusable decapsulation slots temporarily contain sensitive cryptographic material, temporary buffers and slot contents are securely cleared before memory regions are returned to the reusable pool. Slot acquisition, reuse, and release operations are also designed to avoid secret-dependent control flow related to pool management behavior. This reusable lifecycle-oriented design simplifies predictable and secure handling of temporary decapsulation-related memory regions.

5. Limitations and Future Work

5.1 Current Limitations

Although the implementation was evaluated using multiple correctness, robustness, and timing-oriented testing methodologies, several limitations remain. Validation was performed primarily on x86-64 systems using GCC and Clang toolchains, and additional evaluation across other architectures remains desirable.

The implementation also does not currently employ formal constant-time verification methodologies. Timing-related behavior was evaluated using duddect-based statistical leakage testing[5], but broader side-channel analysis may still be beneficial.

In addition, the current implementation prioritizes portability-oriented ISO C11 design principles over aggressive architecture-specific optimizations and therefore does not include assembly-oriented or SIMD-specific optimization modules.

Finally, the reusable decapsulation pool architecture requires additional preallocated memory resources in order to maintain reusable fixed-size decapsulation slots.

5.2 Future Work

Future work may include additional validation across ARM and RISC-V architectures, extended benchmarking against other ML-KEM implementations, and broader side-channel evaluation methodologies. The modular organization of the implementation may also simplify future integration of optional SIMD-oriented or assembly-level optimization modules for selected hardware platforms.

Additional future directions include extended Linux kernel integration, adaptation to embedded or RTOS-oriented environments, and scalability evaluation of larger reusable decapsulation pool configurations.

6. Conclusion

This work presented a portable ISO C11 implementation of the ML-KEM post-quantum key encapsulation mechanism with an emphasis on architectural clarity, explicit memory management, portability-oriented design, and reusable decapsulation resource management.

The implementation employs a reusable pool-based decapsulation architecture intended to reduce repeated allocation and reinitialization overhead during decapsulation operations through fixed-size preallocated reusable slots.

The implementation was evaluated using multiple correctness, robustness, and timing-oriented testing methodologies, including NIST Known Answer Tests (KATs), stress testing, malformed ciphertext validation, ASAN, TSAN, Valgrind, and duddect-based timing leakage analysis under GCC and Clang toolchains on x86-64 systems.

The project is intended as a portable engineering-oriented ML-KEM implementation and experimental platform for further evaluation of reusable cryptographic memory architectures, portability-oriented ISO C11 design strategies, and practical constant-time testing workflows.

References

[1] National Institute of Standards and Technology (NIST), *Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203)*, Federal Information Processing Standards Publication 203, 2024.

<https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.203.pdf>

[2] National Institute of Standards and Technology (NIST), *ACVP Server Test Vectors Repository*, GitHub repository, Available at:

<https://github.com/usnistgov/ACVP-Server/tree/master/gen-val/json-files>

Accessed: March 2026.

[3] International Organization for Standardization (ISO), *ISO/IEC 9899:2011 — Programming Languages — C*, 2011.

<https://www.iso.org/standard/57853.html>

[4] K. Zavertailo, *ML-KEM: Portable ISO C11 Implementation with Reusable Decapsulation Pools*, GitHub repository, 2026.

<https://github.com/kstzv/ml-kem>

[5] duedct project contributors, *duedct: Dude, is my code constant time?*, GitHub repository, 2017.

<https://github.com/oreparaz/duedct>

[6] LLVM Project, Clang AddressSanitizer Documentation,

<https://clang.llvm.org/docs/AddressSanitizer.html>

[7] GNU Project, *GNU Compiler Collection (GCC)*, Official project website. Available at:

<https://gcc.gnu.org/>

[8] Valgrind Developers, *Valgrind Documentation*,

<https://valgrind.org/docs/manual/manual.html>